INVENTORY MANAGEMENT SYSTEM

21CSC205 - DBMS PROJECT REPORT

Submitted by

A.VISHNU VARDHAN REDDY [RA2311030010111]

M.NIKHIL MANIKANTA [RA2311030010113]

Under the Guidance of

Mr.J.PRABAKARAN

Assistant professor, Department of Networking and Communications

in partial fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE ENGINEERING

with specialization in cyber security

DEPARTMENT OF NETWORKING AND COMMUNICATIONS

SCHOOL OF COMPUTING

COLLEGE OF ENGINEERING AND TECHNOLOGY

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

KATTANKULATHUR- 603 203

MAY 2025

Department of Networking and Communications

SRM Institute of Science & Technology

Own Work Declaration Form

Degree/ Course : B.TECH/ CSE W/S CYBERSECURITY

Student Name : A.VISHNU VARDHAN REDDY, M.NIKHIL MANIKANTA

Registration Number :RA2311030010111,RA2311030010113

Title of Work : INVENTORY MANAGEMENT SYSTEM

We hereby certify that this assessment compiles with the University's Rules and Regulations relating to Academic misconduct and plagiarism, as listed in the University Website, Regulations, and the Education Committee guidelines.

We confirm that all the work contained in this assessment is my / our own except where indicated, and that We have met the following conditions:

- Clearly referenced / listed all sources as appropriate
- Referenced and put in inverted commas all quoted text (from books, web, etc)
- Given the sources of all pictures, data etc. that are not my own
- Not made any use of the report(s) or essay(s) of any other student(s) either past or present
- Acknowledged in appropriate places any help that I have received from others (e.g. fellow students, technicians, statisticians, external sources)
- Compiled with any other plagiarism criteria specified in the Course handbook / University website

we understand that any false claim for this work will be penalized in accordance with the University policies and regulations.

DECLARATION:
We are aware of and understand the University's policy on Academic misconduct and plagiarism and I certify that this assessment is our own work, except where indicated by referring, and that I have followed the good academic practices noted above. Reg.no:RA2311030010111 Reg.no:RA2311030010113
Date:

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

KATTANKULATHUR – 603 203

BONAFIDE CERTIFICATE

Certified that 21CSC205P – DBMS Project report titled "**INVENTORY MANAGEMENT SYSTEM"** is the bonafide work of "**A.VISHNU VATDHAN REDDY [RA2311030010111], M.NIKHIL MANIKANTA[RA2311030010113]"** who carried out the project work[internship] under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Mr. Prabakaran J

SUPERVISOR

Assistant professor

Department of Networking and communications

SIGNATURE

Dr. M. Lakshmi

PROFESSOR &HEAD

Department of Networking and communications

ACKNOWLEDGEMENTS

We express our humble gratitude to Dr. C. Muthamizhchelvan, Vice-Chancellor, SRM Institute of Science and Technology, for the facilities extended for the project work and his continued support.

We extend our sincere thanks to Dr. Leenus Jesu Martin M, Dean-CET, SRM Institute of Science and Technology, for his invaluable support.

We wish to thank Dr. Revathi Venkataraman, Professor and Chairperson, School of Computing, SRM Institute of Science and Technology, for her support throughout the project work.

We encompass our sincere thanks to Dr. M. Pushpalatha, Professor and Associate Chairperson - CS, School of Computing and Dr. Lakshmi, Professor and Associate Chairperson -AI, School of Computing, SRM Institute of Science and Technology, for their invaluable support.

We are incredibly grateful to our Head of the Department, Dr Lakshmi M , SRM Institute of Science and Technology, for her suggestions and encouragement at all the stages of the project work.

We register our immeasurable thanks to our Faculty Advisor, Dr. Lakshminarayanan R, Department of Networking and Communication, SRM Institute of Science and Technology, for leading and helping us to complete our course.

Our inexpressible respect and thanks to our guide, Mr. J. Prabakaran, Department of Networking and Communications,

SRM Institute of Science and Technology, for providing us with an opportunity to pursue our project under his mentorship. He provided us with the freedom and support to explore the research topics of our interest. His passion for solving problems and making a difference in the world has always been inspiring.

We sincerely thank all the staff members of Department of Networking and Communication, School of Computing, S.R.M Institute of Science and Technology, for their help during our project. Finally, we would like to thank our parents, family members, and friends for their unconditional love, constant support and encouragement.

Chapter No.	Chapter Name	Page No.
1.	Problem understanding ,Identification of Entity and Relationships,Construction of DB using ER Model for the project.	6- 7
2.	Design of Relational Schemas,Creation of Database Tables for the project.	8 – 14
3.	Complex queries based on the concepts of constraints,sets,joins,views,Triggers and Cursor.	14 – 35
4.	Analyzing the pitfalls, identifying the dependencies ,and applying normalizations	36 – 37
5.	Implementation of concurrency control and recovery mechanisms	37 - 38
6.	Code for the project	39 - 52
7.	Conclusion &Reference	53

AIM OF THE PROJECT

To design and implement a relational database management system (RDBMS) for an inventory management system that efficiently tracks and manages products, stock levels, suppliers, warehouses, orders, shipments, customers, and payments, ensuring accuracy, real-time updates, and ease of data retrieval.

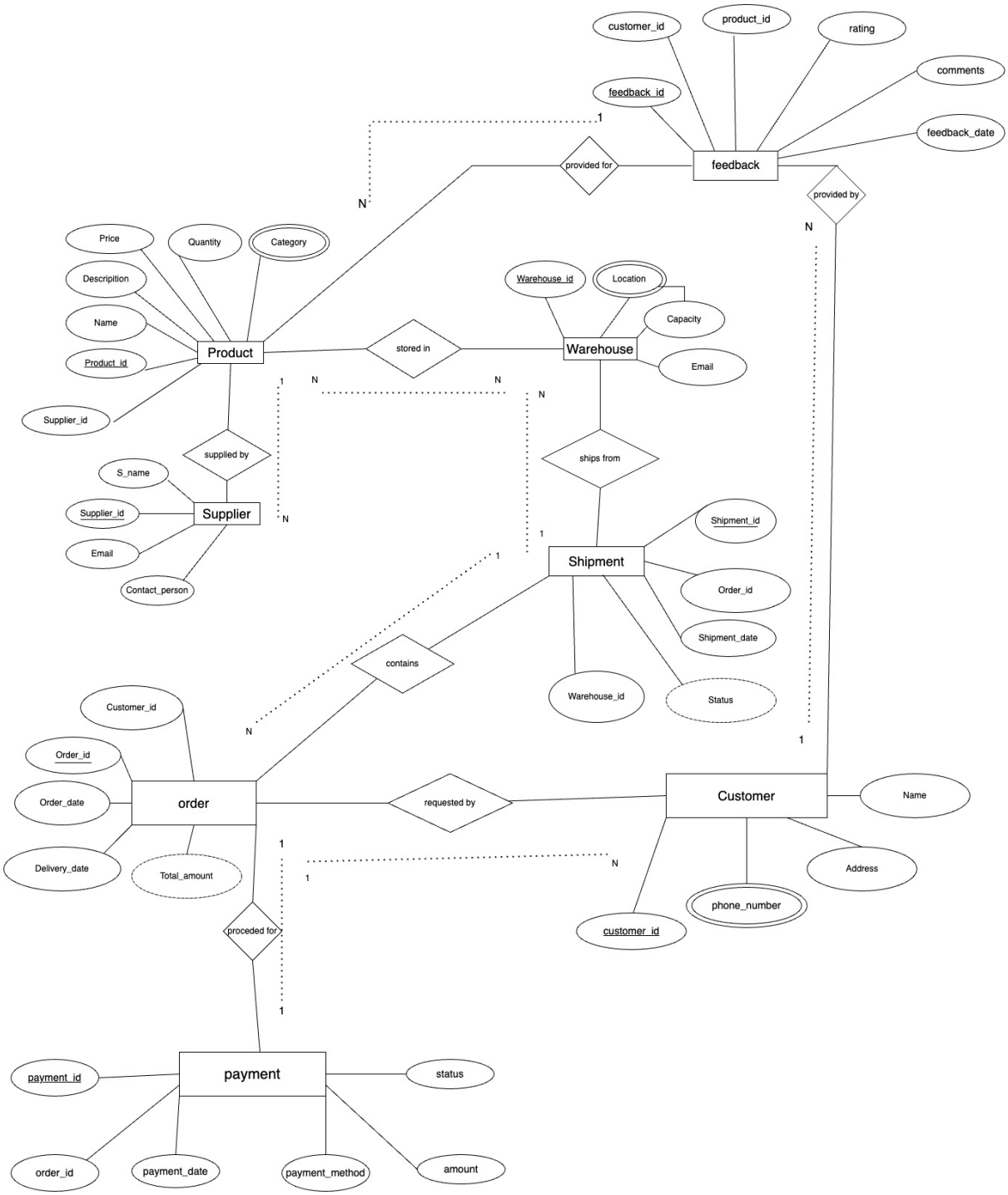
PROBLEM STATEMENT

Traditional inventory systems often suffer from data inconsistencies, lack of real-time visibility, and difficulty scaling with increasing product and transaction volumes. Manual tracking or poorly designed systems can lead to stockouts, overstocking, delayed shipments, and poor supplier/customer communication. Furthermore, the absence of a centralized relational database limits efficient reporting and decision-making.

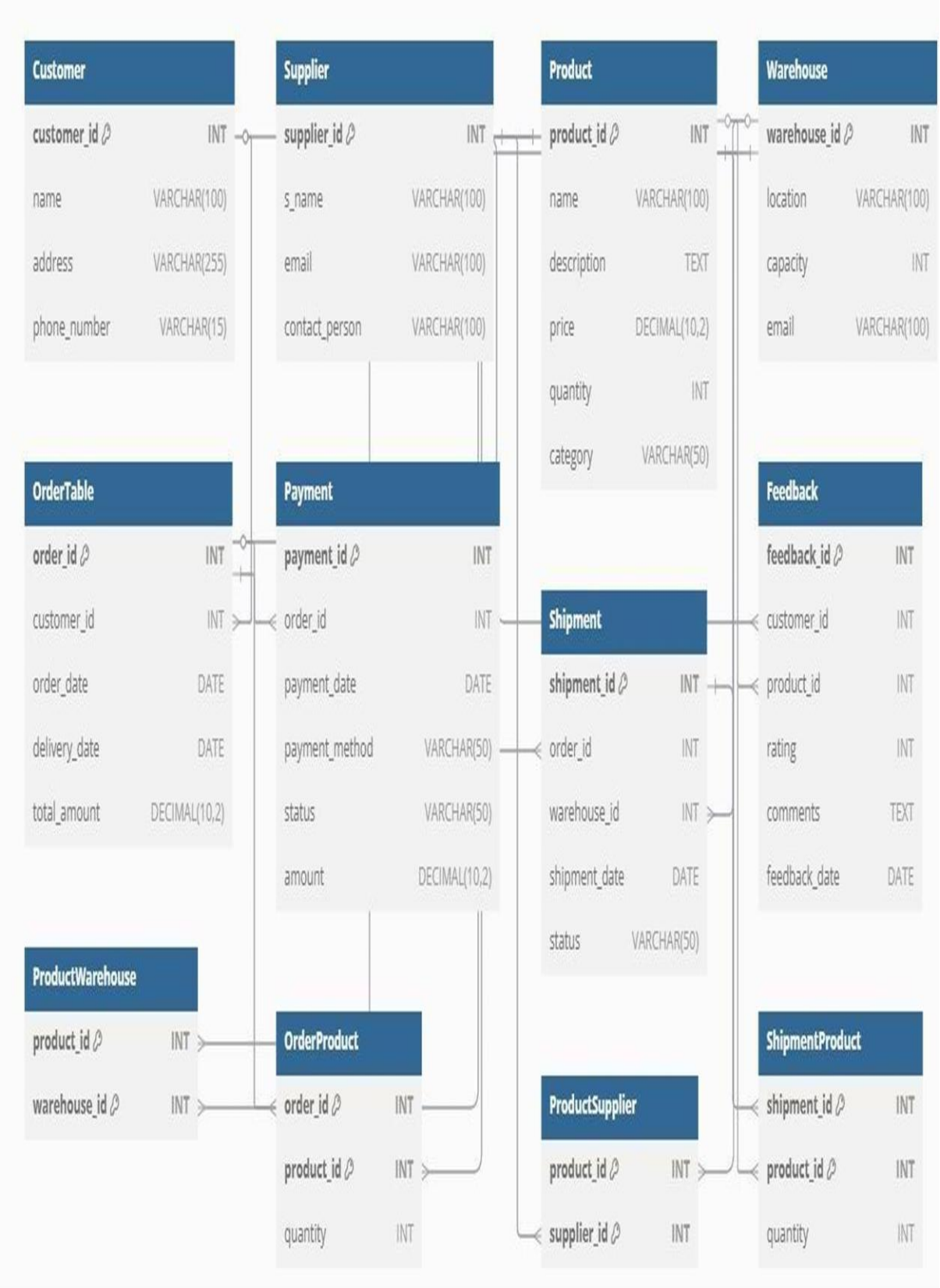
INNOVATIVE ASPECT OF PROJECT

- **Normalized Relational Model** – Ensures data integrity and minimizes redundancy.
- **Real-Time Stock Monitoring** – Enables up-to-date inventory tracking across warehouses.
- **Supplier and Customer Integration** – Tracks full transaction histories with key stakeholders.
- **Automated Order and Shipment Tracking** – Enhances logistics efficiency.
- **Multi-Warehouse Support** – Manages distributed inventory locations with ease.
- **Data Analytics Ready** – Supports future integration with BI tools for demand forecasting and trend analysis.

ER-DIAGRAM:


customer_id
product_id
rating
comments
feedback_id
feedback_date
provided for
feedback
provided by
Price
Quantity
Category
Description
Name
Product_id
Supplier_id
Product
stored in
Warehouse_id
Location
Capacity
Warehouse
Email
supplied by
ships from
S_name
Supplier_id
Supplier
Email
Contact_person
Shipment_id
Shipment
Order_id
Shipment_date
contains
Warehouse_id
Status
Customer_id
Order_id
Order_date
order
requested by
Customer
Name
Delivery_date
Total_amount
Address
phone_number
customer_id
proceded for
payment_id
payment
status
order_id
payment_date
payment_method
amount

SCHEMA DIAGRAM:



Project Description:

The Inventory Management System is a database-driven project designed to manage and track products, customers, suppliers, orders, payments, shipments, warehouses, and customer feedback. It simulates the real-world workflow of a business, allowing for efficient inventory tracking, order processing, and supplier management.

Problem Statement:

Managing inventory, suppliers, customers, and order processing manually or through basic spreadsheets leads to errors, inefficiencies, stock mismatches, and delays in fulfilling customer orders. Businesses often struggle with real-time tracking of product availability, warehouse space, order status, and payment records.

There is a need for a centralized database system that can:

- Efficiently manage inventory across multiple warehouses
- Track customer orders and payments in real-time
- Maintain accurate supplier and product records
- Ensure timely shipments and capture customer feedback
- Automate repetitive tasks like stock updates and data logs using triggers and cursors

Customer Table:

```
CREATE TABLE Customer (
   customer_id INT PRIMARY KEY,
   name VARCHAR(100),
   address VARCHAR(255),
   phone_number VARCHAR(15)
);
```

customer_id	name	address	phone_number
1	Alice	New York	1234567890
2	Bob	Los Angeles	9876543210
3	Charlie	Chicago	4567891230
NULL	NULL	NULL	NULL

Supplier Table:

```
CREATE TABLE Supplier (
    supplier_id INT PRIMARY KEY,
    s_name VARCHAR(100),
    email VARCHAR(100),
    contact_person VARCHAR(100)
);
```

supplier_id	s_name	email	contact_person
1	ABC Supplies	abc@supply.com	John Doe
2	XYZ Traders	xyz@traders.com	Jane Smith
3	GlobalMart	global@mart.com	Steve Adams
NULL	NULL	NULL	NULL

Product Table:

```
CREATE TABLE Product (
    product_id INT PRIMARY KEY,
    name VARCHAR(100),
    description TEXT,
    price DECIMAL(10,2),
```

```
    quantity INT,
    category VARCHAR(50)
);
```

product_id	name	description	price	quantity	category
1	Laptop	Gaming Laptop	1200.00	10	Electronics
2	Headphones	Noise Cancelling	150.00	25	Electronics
3	Chair	Ergonomic Office Chair	95.00	40	Furniture
4	Mouse	Wireless Mouse	25.00	30	Electronics
NULL	NULL	NULL	NULL	NULL	NULL

Warehouse Table:

```
CREATE TABLE Warehouse (
    warehouse_id INT PRIMARY KEY,
    location VARCHAR(100),
    capacity INT,
    email VARCHAR(100)
);
```

warehouse_id	location	capacity	email
1	NY Central	1000	nywarehouse@company.com
2	LA Depot	1500	ladepot@company.com
3	Chicago Hub	1200	chicagohub@company.com
NULL	NULL	NULL	NULL

Order Table:

```
CREATE TABLE OrderTable (
    order_id INT PRIMARY KEY,
    customer_id INT,
    order_date DATE,
```

```
    delivery_date DATE,
    total_amount DECIMAL(10,2),
    FOREIGN KEY (customer_id) REFERENCES Customer(customer_id)
);
```

order_id	customer_id	order_date	delivery_date	total_amount
1	1	2025-03-01	2025-03-10	1250.00
2	2	2025-03-02	2025-03-06	150.00
3	3	2025-03-03	2025-03-07	95.00
NULL	NULL	NULL	NULL	NULL

Payment Table:

```
CREATE TABLE Payment (
    payment_id INT PRIMARY KEY,
    order_id INT,
    payment_date DATE,
    payment_method VARCHAR(50),
    status VARCHAR(50),
    amount DECIMAL(10,2),
    FOREIGN KEY (order_id) REFERENCES OrderTable(order_id)
);
```

payment_id	order_id	payment_date	payment_method	status	amount
1	1	2025-03-01	Credit Card	Completed	1200.00
2	2	2025-03-02	PayPal	Completed	150.00
3	3	2025-03-03	Debit Card	Pending	95.00
NULL	NULL	NULL	NULL	NULL	NULL

Shipment Table:

```
CREATE TABLE Shipment (
    shipment_id INT PRIMARY KEY,
    order_id INT,
    warehouse_id INT,
    shipment_date DATE,
    status VARCHAR(50),
    FOREIGN KEY (order_id) REFERENCES OrderTable(order_id),
    FOREIGN KEY (warehouse_id) REFERENCES Warehouse(warehouse_id)
);
```

shipment_id	order_id	warehouse_id	shipment_date	status
1	1	1	2025-03-04	Shipped
2	2	2	2025-03-05	Delivered
3	3	3	2025-03-06	In Transit
NULL	NULL	NULL	NULL	NULL

Feedback Table:

```
CREATE TABLE Feedback (
    feedback_id INT PRIMARY KEY,
    customer_id INT,
    product_id INT,
    rating INT,
    comments TEXT,
    feedback_date DATE,
    FOREIGN KEY (customer_id) REFERENCES Customer(customer_id),
    FOREIGN KEY (product_id) REFERENCES Product(product_id)
```

);

feedback_id	customer_id	product_id	rating	comments	feedback_date
1	1	1	5	Excellent performance	2025-03-06
2	2	2	4	Good quality sound	2025-03-07
3	3	3	3	Comfortable but average quality	2025-03-08
NULL	NULL	NULL	NULL	NULL	NULL

SIMPLE QUERIES:

1. List all customers

SELECT * FROM Customer;

customer_id	name	address	phone_number
1	Alice	New York	1234567890
2	Bob	Los Angeles	9876543210
3	Charlie	Chicago	4567891230
NULL	NULL	NULL	NULL

2. Find all products in the "Electronics" category

SELECT * FROM Product

WHERE category = 'Electronics';

product_id	name	description	price	quantity	category
1	Laptop	Gaming Laptop	1200.00	10	Electronics
2	Headphones	Noise Cancelling	150.00	25	Electronics
4	Mouse	Wireless Mouse	25.00	30	Electronics
NULL	NULL	NULL	NULL	NULL	NULL

3. Show all orders placed by customer with ID = 1

SELECT * FROM OrderTable

WHERE customer_id = 1;

	order_id	customer_id	order_date	delivery_date	total_amount
▶	1	1	2025-03-01	2025-03-10	1250.00
*	NULL	NULL	NULL	NULL	

4. Show all shipments that are marked as "Delivered"

SELECT * FROM Shipment

WHERE status = 'Delivered';

	shipment_id	order_id	warehouse_id	shipment_date	status
▶	2	2	2	2025-03-05	Delivered
*	NULL	NULL	NULL	NULL	NULL

S

5. Find products priced above ₹500

SELECT * FROM Product

WHERE price > 500;

	product_id	name	description	price	quantity	category
▶	1	Laptop	Gaming Laptop	1200.00	10	Electronics
*	NULL	NULL	NULL	NULL	NULL	NULL

COMPLEX QUERIES:

1. Get order details with customer name

```
SELECT o.order_id, c.name AS customer_name, o.total_amount
FROM OrderTable o
JOIN Customer c ON o.customer_id = c.customer_id;
```

order_id	customer_name	total_amount
1	Alice	1250.00
2	Bob	150.00
3	Charlie	95.00

2. List all products along with their suppliers

```
SELECT p.name AS product_name, s.s_name AS supplier_name
FROM Product p
JOIN ProductSupplier ps ON p.product_id = ps.product_id
JOIN Supplier s ON ps.supplier_id = s.supplier_id;
```

product_name	supplier_name
Laptop	ABC Supplies
Headphones	XYZ Traders
Chair	GlobalMart

3. Get total sales per customer

```
SELECT c.name, SUM(o.total_amount) AS total_spent
```

```
FROM Customer c
JOIN OrderTable o ON c.customer_id = o.customer_id
GROUP BY c.name;
```

	name	total_spent
▶	Alice	1250.00
	Bob	150.00
	Charlie	95.00

4. Find customers who gave feedback with rating 5

```
SELECT DISTINCT c.name
FROM Feedback f
JOIN Customer c ON f.customer_id = c.customer_id
WHERE f.rating = 5;
```

	name
▶	Alice

5. Show order details along with product names

```
SELECT o.order_id, p.name AS product_name, op.quantity
FROM OrderTable o
JOIN OrderProduct op ON o.order_id = op.order_id
JOIN Product p ON op.product_id = p.product_id;
```

order_id	product_name	quantity
1	Laptop	1
2	Headphones	2
3	Chair	1

Views with Subqueries:

1) Total number of orders per customer

A) CREATE VIEW CustomerOrderCount AS

SELECT c.customer_id, c.name,

(SELECT COUNT(*) FROM OrderTable o WHERE o.customer_id = c.customer_id) AS order_count

FROM Customer c;

customer_id	name	order_count
1	Alice	1
2	Bob	1
3	Charlie	1

2. Product stock across warehouses

A) CREATE VIEW ProductWarehouseView AS

SELECT p.product_id, p.name, w.location, p.quantity

FROM Product p

JOIN ProductWarehouse pw ON p.product_id = pw.product_id

JOIN Warehouse w ON pw.warehouse_id = w.warehouse_id;

product_id	name	location	quantity
1	Laptop	NY Central	10
2	Headphones	LA Depot	25
3	Chair	Chicago Hub	40

3. Total payment amount per customer

A) CREATE VIEW CustomerPaymentTotal AS

SELECT c.customer_id, c.name,

(SELECT SUM(p.amount)

FROM Payment p

JOIN OrderTable o ON o.order_id = p.order_id

WHERE o.customer_id = c.customer_id) AS total_payment

FROM Customer c;

customer_id	name	total_payment
1	Alice	1200.00
2	Bob	150.00
3	Charlie	95.00

4. Product feedback summary

A) CREATE VIEW ProductFeedback AS

SELECT p.product_id, p.name,

(SELECT AVG(rating) FROM Feedback f WHERE f.product_id = p.product_id) AS avg_rating

FROM Product p;

	product_id	name	avg_rating
▶	1	Laptop	5.0000
	2	Headphones	4.0000
	3	Chair	3.0000

5. Orders with shipment status

A) CREATE VIEW OrderShipmentStatus AS

SELECT o.order_id, o.order_date,

(SELECT status FROM Shipment s WHERE s.order_id = o.order_id) AS shipment_status

FROM OrderTable o;

	order_id	order_date	shipment_status
▶	1	2025-03-01	Shipped
	2	2025-03-02	Delivered
	3	2025-03-03	In Transit

6. Orders with payment status

A) CREATE VIEW OrderPaymentStatus AS

SELECT o.order_id, o.customer_id, o.total_amount,

(SELECT status FROM Payment p WHERE p.order_id = o.order_id) AS payment_status

FROM OrderTable o;

	order_id	customer_id	total_amount	payment_status
▶	1	1	1200.00	Completed
	2	2	150.00	Completed
	3	3	95.00	Pending

7. Customer feedback summary

A) CREATE VIEW CustomerFeedbackSummary AS

SELECT c.customer_id, c.name,

(SELECT COUNT(*) FROM Feedback f WHERE f.customer_id = c.customer_id) AS total_feedbacks

FROM Customer c;

	customer_id	name	total_feedbacks
▶	1	Alice	1
	2	Bob	1
	3	Charlie	1

8. Top-rated products (rating >= 4)

A) CREATE VIEW TopRatedProducts AS

SELECT p.product_id, p.name

FROM Product p

WHERE (SELECT AVG(rating) FROM Feedback f WHERE f.product_id = p.product_id) >= 4;

	product_id	name
▶	1	Laptop
	2	Headphones

9. Products with low stock (< 10)

A) CREATE VIEW LowStockProducts AS

SELECT product_id, name, quantity

FROM Product

WHERE quantity < 10;

	product_id	name	quantity

NOTE:As there is no product with less quantity than 10 hence the data is NULL

10. Latest order per customer

A) CREATE VIEW LatestOrderPerCustomer AS

SELECT c.customer_id, c.name,

(SELECT MAX(order_date) FROM OrderTable o WHERE o.customer_id = c.customer_id) AS last_order_date

FROM Customer c;

customer_id	name	last_order_date
1	Alice	2025-03-01
2	Bob	2025-03-02
3	Charlie	2025-03-03

Join Queries:

1. List all orders with customer names

A) SELECT o.order_id, c.name, o.total_amount

FROM OrderTable o

JOIN Customer c ON o.customer_id = c.customer_id;

order_id	name	total_amount
1	Alice	1200.00
2	Bob	150.00
3	Charlie	95.00

2. Payment details with order and customer info

A) SELECT p.payment_id, o.order_id, c.name, p.amount, p.status

FROM Payment p

JOIN OrderTable o ON p.order_id = o.order_id

JOIN Customer c ON o.customer_id = c.customer_id;

payment_id	order_id	name	amount	status
1	1	Alice	1200.00	Completed
2	2	Bob	150.00	Completed
3	3	Charlie	95.00	Pending

3. Shipments with warehouse info

A) SELECT s.shipment_id, w.location, s.status

FROM Shipment s

JOIN Warehouse w ON s.warehouse_id = w.warehouse_id;

shipment_id	location	status
1	NY Central	Shipped
2	LA Depot	Delivered
3	Chicago Hub	In Transit

4. Product details with supplier name

A) SELECT p.name AS product, s.s_name AS supplier

FROM Product p

JOIN ProductSupplier ps ON p.product_id = ps.product_id

JOIN Supplier s ON ps.supplier_id = s.supplier_id;

	product	supplier
▶	Laptop	ABC Supplies
	Headphones	XYZ Traders
	Chair	GlobalMart

5. Feedback with customer and product info

A) SELECT f.feedback_id, c.name, p.name AS product, f.rating

FROM Feedback f

JOIN Customer c ON f.customer_id = c.customer_id

JOIN Product p ON f.product_id = p.product_id;

	feedback_id	name	product	rating
▶	1	Alice	Laptop	5
	2	Bob	Headphones	4
	3	Charlie	Chair	3

6. Products in orders

A) SELECT o.order_id, p.name AS product_name, op.quantity

FROM OrderProduct op

JOIN OrderTable o ON op.order_id = o.order_id

JOIN Product p ON op.product_id = p.product_id;

order_id	product_name	quantity
1	Laptop	1
2	Headphones	2
3	Chair	1

7. Products in shipments

A) SELECT s.shipment_id, p.name, sp.quantity

FROM ShipmentProduct sp

JOIN Shipment s ON sp.shipment_id = s.shipment_id

JOIN Product p ON sp.product_id = p.product_id;

shipment_id	name	quantity
1	Laptop	1
2	Headphones	2
3	Chair	1

8. All warehouses with their products

A) SELECT w.location, p.name AS product

FROM ProductWarehouse pw

JOIN Warehouse w ON pw.warehouse_id = w.warehouse_id

JOIN Product p ON pw.product_id = p.product_id;

	location	product
▶	NY Central	Laptop
	LA Depot	Headphones
	Chicago Hub	Chair

9. Orders with products and customer name

```
SELECT o.order_id, c.name AS customer, p.name AS product, op.quantity
FROM OrderTable o
JOIN Customer c ON o.customer_id = c.customer_id
JOIN OrderProduct op ON o.order_id = op.order_id
JOIN Product p ON op.product_id = p.product_id;
```

	order_id	customer	product	quantity
▶	1	Alice	Laptop	1
	2	Bob	Headphones	2
	3	Charlie	Chair	1

10. Shipments with order and customer info

A)
```
SELECT s.shipment_id, s.status, o.order_id, c.name AS customer
FROM Shipment s
JOIN OrderTable o ON s.order_id = o.order_id
JOIN Customer c ON o.customer_id = c.customer_id;
```

	shipment_id	status	order_id	customer
▶	1	Shipped	1	Alice
	2	Delivered	2	Bob
	3	In Transit	3	Charlie

Set Operations:

1. Products in orders or in shipments (union)

A) SELECT product_id FROM OrderProduct

UNION

SELECT product_id FROM ShipmentProduct;

	product_id
▶	1
	2
	3

2. Products both ordered and shipped (intersect)

A) SELECT product_id FROM OrderProduct

INTERSECT

SELECT product_id FROM ShipmentProduct;

product_id
1
2
3

3. Products ordered but not shipped (except)

A) SELECT product_id FROM OrderProduct

EXCEPT

SELECT product_id FROM ShipmentProduct;

product_id

4. Customers who gave feedback or placed orders

A) SELECT customer_id FROM Feedback

UNION

SELECT customer_id FROM OrderTable;

customer_id
1
2
3

5. Customers who placed orders but didn’t give feedback

A) SELECT customer_id FROM OrderTable

EXCEPT

SELECT customer_id FROM Feedback;

	customer_id

6. Warehouses that have shipped products or stored products

A) SELECT warehouse_id FROM Shipment

UNION

SELECT warehouse_id FROM ProductWarehouse;

	warehouse_id
▶	1
	2
	3

7. Warehouses that stored products but didn't ship

A) SELECT warehouse_id FROM ProductWarehouse

EXCEPT

SELECT warehouse_id FROM Shipment;

	warehouse_id

8. Orders that have both payment and shipment

A) SELECT order_id FROM Payment

INTERSECT

SELECT order_id FROM Shipment;

	order_id
▶	1
	2
	3

9. Orders with payment but no shipment

A) SELECT order_id FROM Payment

EXCEPT

SELECT order_id FROM Shipment;

	order_id

10. Products supplied by a supplier but never ordered

A) SELECT product_id FROM ProductSupplier

EXCEPT

SELECT product_id FROM OrderProduct;

	product_id

CURSOR:

```
DELIMITER $$

CREATE PROCEDURE ShowCompletedPayments()
BEGIN
   -- Declare variables
   DECLARE done INT DEFAULT FALSE;
   DECLARE pid INT;
   DECLARE amt DECIMAL(10,2);

   -- Declare cursor for completed payments
   DECLARE payment_cursor CURSOR FOR
      SELECT payment_id, amount FROM Payment WHERE status =
'Completed';

   -- Handle end of data
   DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

   -- Open the cursor
   OPEN payment_cursor;

   read_loop: LOOP
      FETCH payment_cursor INTO pid, amt;

      -- Exit when done
```

```
        IF done THEN
            LEAVE read_loop;
        END IF;

        -- Do something with each row (for example, print it)
        SELECT CONCAT('Payment ID: ', pid, ', Amount: ', amt) AS Output;

    END LOOP;

    -- Close the cursor
    CLOSE payment_cursor;
END$$

DELIMITER ;
```

	Output
▶	Payment ID: 2, Amount: 150.00

TRIGGER:

1. BEFORE UPDATE Trigger for PAYMENT

```
DELIMITER //

CREATE TRIGGER before_payment_update
BEFORE UPDATE ON Payment
FOR EACH ROW
```

```
BEGIN
   IF NEW.amount < 0 THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Payment amount cannot be negative';
   END IF;

   IF NEW.status NOT IN ('Completed', 'Pending', 'Failed') THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Invalid payment status';
   END IF;
END;
//

DELIMITER ;
```

2. AFTER UPDATE Trigger for PAYMENT

```
DELIMITER //

CREATE TRIGGER after_payment_update
AFTER UPDATE ON Payment
FOR EACH ROW
BEGIN
   INSERT INTO PaymentUpdateLog (
      payment_id,
      old_status, new_status,
```

```
        old_amount, new_amount
    )
    VALUES (
        NEW.payment_id,
        OLD.status, NEW.status,
        OLD.amount, NEW.amount
    );
END;
//

DELIMITER ;
```

Key Design Pitfalls Identified

1. Stock Quantity Stored in Product Table

- **Issue:** The Product table has a quantity column, but inventory should be managed **per location**, not globally.
- **Fix:** Remove quantity from Product and instead manage stock using a Stock or ProductWarehouse table with a quantity field.

Recommended Structural Improvements

1. **Normalize units** – Consider a Units table if you sell in different formats (e.g., pieces, boxes).
2. **SupplierProductPrice** – If suppliers offer different prices, track them per product-supplier pair.
3. **Separate category entity** – Instead of using VARCHAR, use a Category table.

Functional Dependencies

Functional dependencies (FDs) describe the relationship between attributes in a relational database and are key for **normalization** and **data integrity**. Let's analyze the **functional dependencies** in your **inventory management schema**, table by table.

Normalization Summary & Improvements by Table

1. **Product Table**

```
CREATE TABLE Category (
  category_id INT PRIMARY KEY,
  name VARCHAR(100)
);

ALTER TABLE Product DROP COLUMN quantity, DROP COLUMN category;
ALTER TABLE Product ADD category_id INT,
  FOREIGN KEY (category_id) REFERENCES Category(category_id);
```

2. **ProductWarehouse Table**

ProductWarehouse(product_id, warehouse_id)

ALTER TABLE ProductWarehouse ADD quantity INT;

-- (product_id, warehouse_id) → quantity

Optional Normalized Tables to Add

Units

```
CREATE TABLE Unit (
  unit_id INT PRIMARY KEY,
  name VARCHAR(50) -- e.g., 'piece', 'box', 'kg'
);
```

OrderStatus

```
CREATE TABLE OrderStatus (
status_id INT PRIMARY KEY,
description VARCHAR(50)
);
```

Optimistic Concurrency Control:

Optimistic Concurrency Control (OCC) is a concurrency management strategy in databases where conflicts are assumed to be **rare**, so transactions proceed without locking resources.

At **commit time**, the system checks whether a conflict occurred. If so, the transaction **rolls back**.

It's ideal for **read-heavy, low-conflict systems**, like inventory management systems with many read operations but relatively few writes.

Add a Version Column

```
ALTER TABLE ProductWarehouse ADD version INT DEFAULT 0;
```

Read & Store Version

```
SELECT quantity, version FROM ProductWarehouse

WHERE product_id = 1 AND warehouse_id = 1;
```

Update Using Version Check

```
UPDATE ProductWarehouse

SET quantity = quantity - 1, version = version + 1

WHERE product_id = 1 AND warehouse_id = 1 AND version = 3;
```

INVENTORY MANAGEMENT SYSTEM FRONTEND CODE ANALYSIS

Overview

The frontend of the Inventory Management System is built using modern web technologies (likely HTML, CSS, and JavaScript frameworks) to provide an intuitive and real-time dashboard interface. It allows users to monitor inventory items, suppliers, stock value, and transactions through a responsive and data-driven UI. The system communicates with the backend through RESTful APIs to retrieve and display inventory and supplier information dynamically.

Architecture

- **HTML**: Structures the dashboard with components such as statistic cards, graphs, and navigation menus.
- **CSS**: Implements a clean, responsive layout ensuring compatibility with various screen sizes.
- **JavaScript**: Manages real-time updates (e.g., Refresh Dashboard), handles UI interactions, fetches data from APIs, and renders visual charts.

Dashboard Features

- **Total Items**: Displays the count of unique inventory items grouped by category.
- **Inventory Value**: Shows the total monetary value of the current inventory.
- **Low Stock Items**: Alerts for items below the minimum stock threshold (threshold: 10 units).
- **Suppliers**: Displays the number of active supplier relationships.
- **Monthly Transaction Values**: Visual graph (currently empty in the screenshot) likely showing monthly purchase/sale values.
- **Stock Value by Category**: Donut chart displaying inventory distribution across categories like Electronics, Furniture, and Office Supplies.

Functionality

- **Real-Time Dashboard**: Automatically or manually refreshes data via the "Refresh Dashboard" button.
- **Inventory Analytics**: Provides summary metrics and visual breakdowns for quick decision-making.
- **Supplier Management**: Displays the number of active partnerships with suppliers.
- **Stock Monitoring**: Tracks stock levels, highlighting items below threshold.

- **Graphical Insights**: Includes visual tools like bar and donut charts for better data interpretation.

Strengths

- **Real-Time Feedback**: Instant refresh option ensures data stays up-to-date.
- **Visual Clarity**: Cleanly designed UI with icons, color-coded statuses, and graphs for immediate understanding.
- **Responsive Layout**: Likely adaptable for different screen sizes and devices.
- **User-Centric Design**: Prioritizes high-level visibility of key metrics for inventory managers.

Conclusion

The frontend offers a professional, informative dashboard interface for managing inventory effectively. With visually organized metrics, real-time updates, and clear graphical representations, the system enables quick assessment and decision-making. It is designed with user efficiency in mind, delivering a smooth and responsive experience for managing stock, suppliers, and overall inventory health.

INVENTORY MANAGEMENT BACKEND CODE ANALYSIS

Overview

The backend is built using **Python and Flask**, structured with modular components like models, forms, routes, and utility functions. It manages items, stock, users, suppliers, and reports, providing a web interface and APIs for inventory-related operations. It's designed for small-to-medium business inventory control with administrative capabilities.

Architecture

- **Flask Framework** handles routing and HTTP request/response lifecycle.
- **Jinja2 Templates** render dynamic HTML views.
- **SQL-based Database (likely PostgreSQL/MySQL)** interacts via model abstraction (models.py).
- **Blueprints/Views** manage each domain (items, suppliers, reports, users).
- **Forms (WTForms)** are used for input validation and user interaction.
- **Session Management** handles authentication, with auto_login() for development.

Database Design

- **Users**: Stores login credentials and roles (admin/user).
- **Items**: Core table containing name, description, stock quantity, price, and category.
- **Suppliers**: Stores supplier contact info and relationships with items.
- **StockTransactions**: Logs stock in and stock out events with timestamps and sources/destinations.
- **Reports**: Uses summary and filter-based data queries across transactions and inventory.
- **Categories**: Implicitly modeled as a field in Item.

Functionality

- **Inventory Management**: Add, update, delete, and search items with stock threshold alerts.
- **Stock In/Out Tracking**: Logs incoming supplies and outgoing issues with date tracking and validation.
- **Supplier Management**: CRUD operations on supplier details.
- **User Management**: Admin-only control of system users.
- **Reports**:
    - Stock summary
    - Monthly transactions
    - Category-wise and supplier-wise analysis
    - Transaction history (in/out)
- **Charts**: Uses JSON-encoded data for frontend visualization (monthly and category summaries).
- **Authentication**: Basic session management with admin_required decorators for access control.
- **REST APIs**: JSON responses for stock level checks and low-stock item counts (used by frontend JavaScript).

Strengths

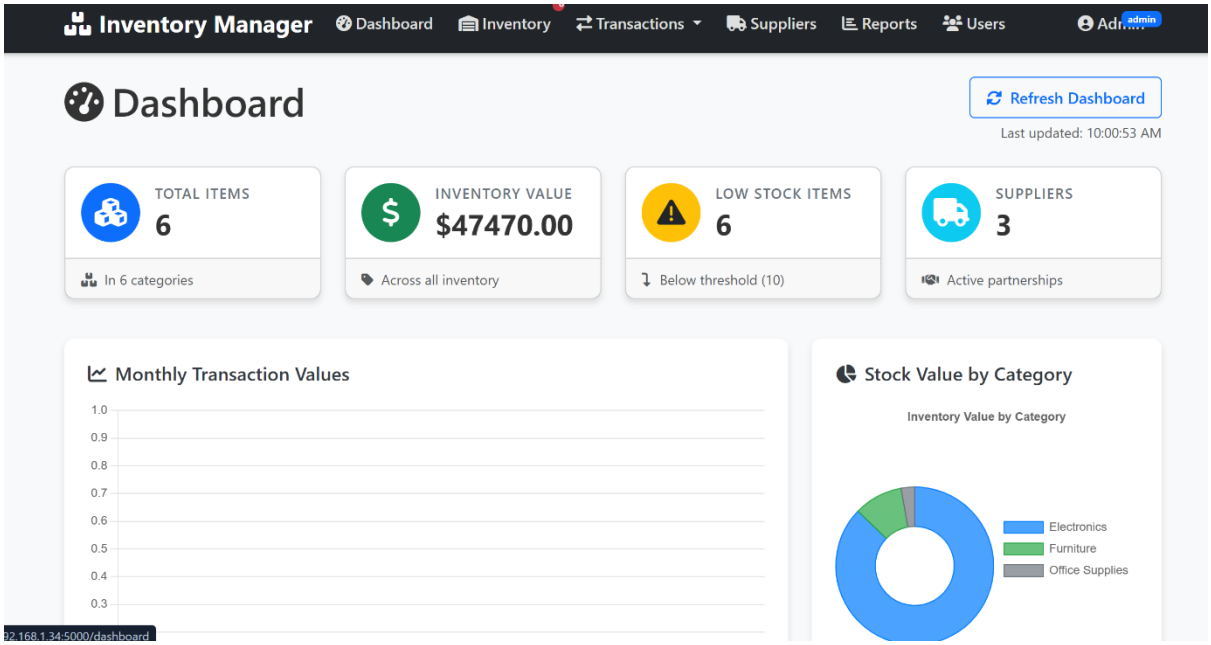
- **Extensible Architecture**: Clear separation of logic (routes, models, utils).
- **Admin Control**: Offers secure role-based access to sensitive operations.
- **Interactive UI**: Uses Flask forms and templates for easy data entry and validation.
- **Comprehensive Reports**: Real-time inventory insights with graphical data presentation.
- **Developer-Friendly**: Auto-login simplifies local development and testing.

Conclusion

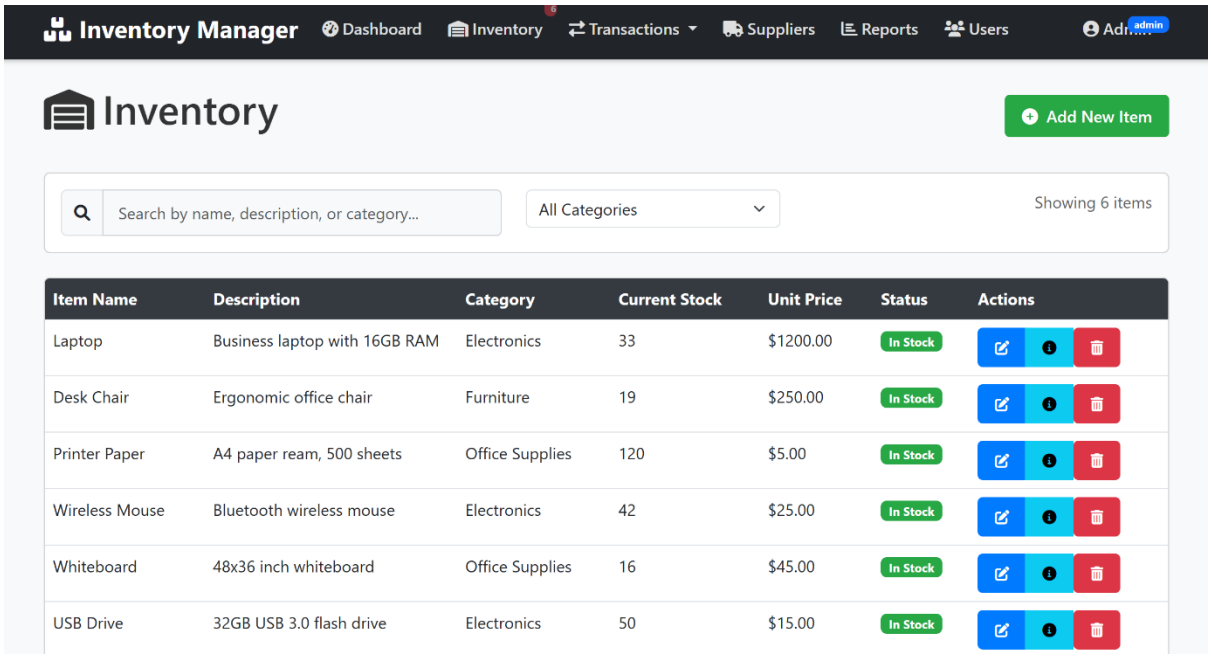
The backend is a well-organized, Flask-powered inventory management system that handles all major aspects of product and stock control. Its modular architecture, built-in reporting, and admin features make it a maintainable and scalable solution suitable for growing inventory operations. It balances simplicity with robust functionality, ideal for inventory systems in retail, warehouse, or office settings.

OUTPUT SCREENSHOTS :

THIS PICTURE SHOWS THE WEBSITE FRONT PAGE

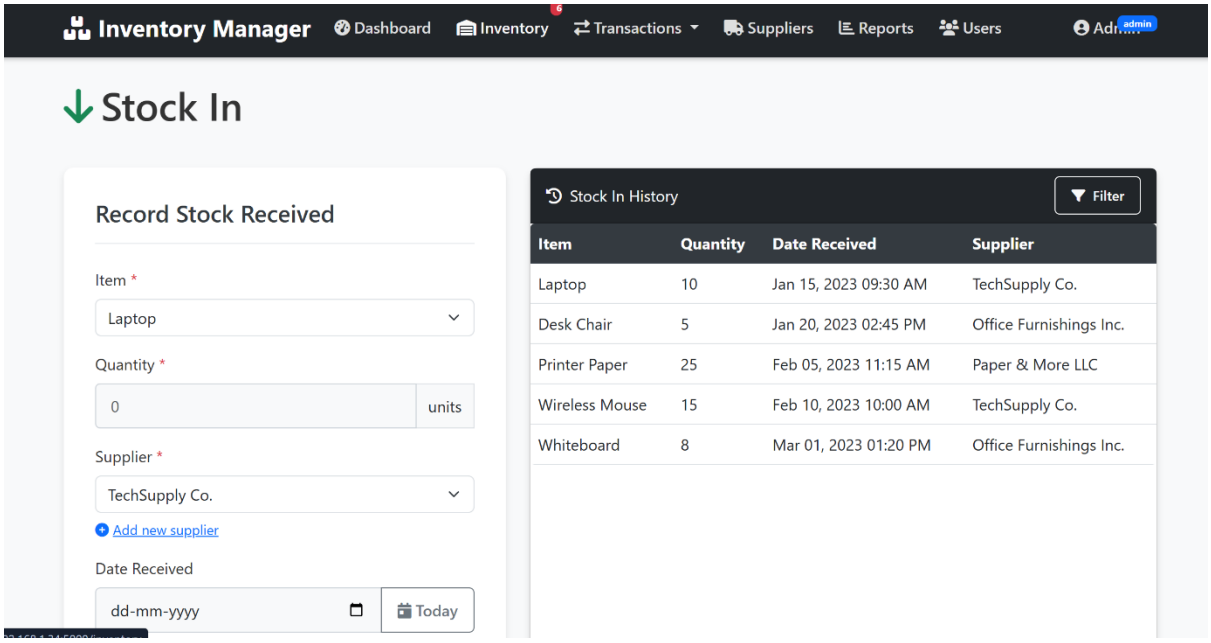


THIS PICTURE SHOWS THE CURRENT STOCK IN THE INVENTORY

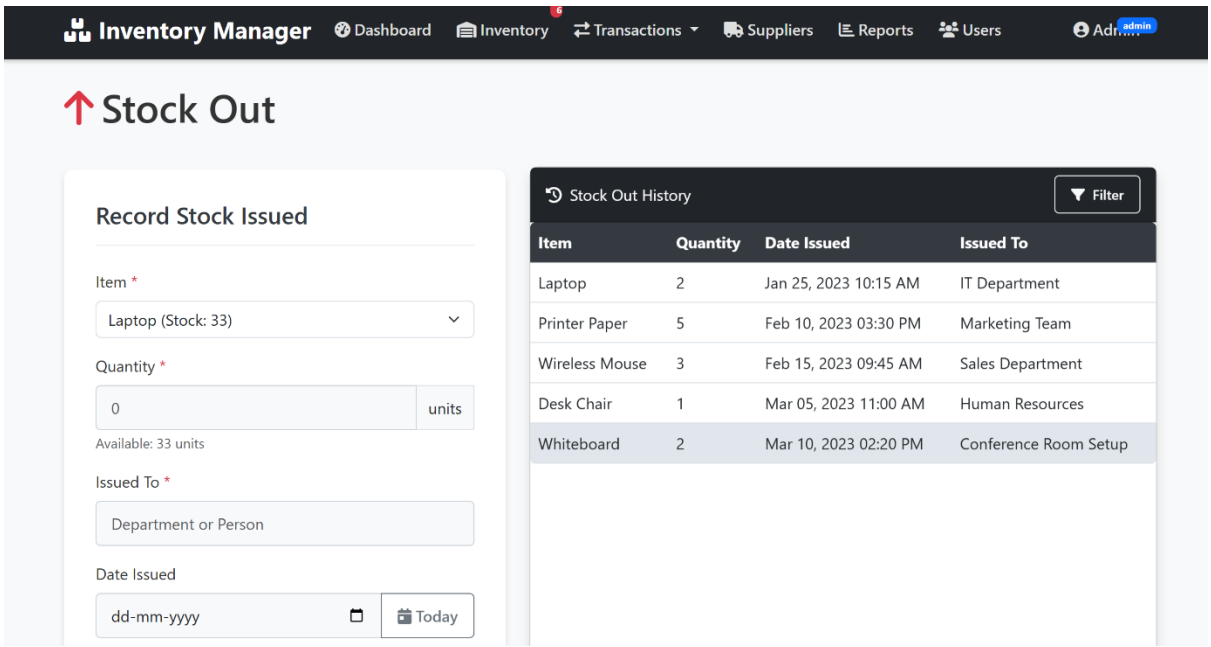


THIS PICTURE SHOWS THE TRANSACTION DETAILS

STOCK IN:



STOCK OUT:



BACKEND CODE:

```
from flask import Flask, render_template, request, redirect, url_for, flash, session, jsonify

from config import Config

from database import Database

from models import User, Item, Supplier, StockTransaction, Report

from forms import (LoginForm, ItemForm, SupplierForm, StockInForm, StockOutForm,

                   UserForm, SearchForm, ReportFilterForm)

from utils import admin_required, format_date, format_currency, calculate_status,
prepare_chart_data

from datetime import datetime, timedelta

from werkzeug.security import generate_password_hash

import json


app = Flask(__name__)

app.config.from_object(Config)


# Register template filters

@app.template_filter('format_date')

def format_date_filter(date_str):

    return format_date(date_str)


@app.template_filter('format_currency')

def format_currency_filter(amount):

    return format_currency(amount)


@app.context_processor

def inject_global_data():

    return {
```

```
        'user_role': session.get('user_role', None),
        'username': session.get('username', None),
        'search_form': SearchForm(),
        'low_stock_threshold': Config.LOW_STOCK_THRESHOLD
    }

@app.before_request
def auto_login():
    # Automatically log in as admin for all routes
    if 'user_id' not in session and request.endpoint != 'static':
        # Set admin user session data
        session['user_id'] = 1  # Assuming admin ID is 1
        session['username'] = 'Admin'
        session['user_role'] = 'admin'
        session.permanent = True

# Root route redirects to dashboard
@app.route('/', methods=['GET'])
def index():
    return redirect(url_for('dashboard'))

@app.route('/logout')
def logout():
    session.clear()
    # Re-login automatically
    session['user_id'] = 1  # Assuming admin ID is 1
    session['username'] = 'Admin'
    session['user_role'] = 'admin'
```

```
    session.permanent = True
    flash('Session has been reset', 'info')
    return redirect(url_for('dashboard'))

# Dashboard
@app.route('/dashboard')
def dashboard():
    # Get summary data for dashboard
    low_stock_items = Item.get_low_stock_items(Config.LOW_STOCK_THRESHOLD)
    recent_transactions = StockTransaction.get_recent_transactions(5)

    # Get counts for dashboard widgets
    stock_summary = Report.get_stock_summary()
    total_items = len(stock_summary) if stock_summary else 0
    total_value = sum(float(item['stock_value'] or 0) for item in stock_summary) if
stock_summary else 0

    categories = Item.get_categories()
    total_categories = len(categories)

    suppliers = Supplier.get_all()
    total_suppliers = len(suppliers) if suppliers else 0

    # Get data for charts
    monthly_data = Report.get_monthly_transactions()
    category_data = Report.get_category_summary()

    # Prepare chart data
```

```
    monthly_chart_data = prepare_chart_data(monthly_data, 'monthly_transactions')
    category_chart_data = prepare_chart_data(category_data, 'category_summary')

    return render_template(
        'dashboard.html',
        low_stock_items=low_stock_items,
        recent_transactions=recent_transactions,
        total_items=total_items,
        total_value=total_value,
        total_categories=total_categories,
        total_suppliers=total_suppliers,
        low_stock_count=len(low_stock_items) if low_stock_items else 0,
        monthly_chart_data=json.dumps(monthly_chart_data) if monthly_chart_data else
None,
        category_chart_data=json.dumps(category_chart_data) if category_chart_data else
None
    )

# Inventory Management Routes
@app.route('/inventory')
def inventory():
    items = Item.get_all()
    return render_template('inventory.html', items=items,
threshold=Config.LOW_STOCK_THRESHOLD)

@app.route('/inventory/add', methods=['GET', 'POST'])
def add_item():
    form = ItemForm()
    if form.validate_on_submit():
```

```
        Item.create(
            name=form.name.data,
            description=form.description.data,
            category=form.category.data,
            quantity=form.quantity.data,
            unit_price=form.unit_price.data
        )
        flash('Item added successfully', 'success')
        return redirect(url_for('inventory'))

    return render_template('item_form.html', form=form, title='Add New Item')

@app.route('/inventory/edit/<int:item_id>', methods=['GET', 'POST'])
def edit_item(item_id):
    item = Item.get_by_id(item_id)
    if not item:
        flash('Item not found', 'danger')
        return redirect(url_for('inventory'))

    form = ItemForm(obj=item)
    if form.validate_on_submit():
        Item.update(
            item_id=item_id,
            name=form.name.data,
            description=form.description.data,
            category=form.category.data,
            quantity=form.quantity.data,
            unit_price=form.unit_price.data
```

```
        )
        flash('Item updated successfully', 'success')
        return redirect(url_for('inventory'))

    return render_template('item_form.html', form=form, title='Edit Item', item=item)

@app.route('/inventory/delete/<int:item_id>', methods=['POST'])
@admin_required
def delete_item(item_id):
    Item.delete(item_id)
    flash('Item deleted successfully', 'success')
    return redirect(url_for('inventory'))

@app.route('/inventory/search')
def search_inventory():
    query = request.args.get('query', '')
    if not query:
        return redirect(url_for('inventory'))

    items = Item.search(query)
    return render_template('inventory.html', items=items, search_query=query,
threshold=Config.LOW_STOCK_THRESHOLD)

# Stock In/Out Routes
@app.route('/stock/in', methods=['GET', 'POST'])
def stock_in():
    form = StockInForm()
```

```
    # Populate select fields
    form.item_id.choices = [(item['id'], item['name']) for item in Item.get_all()]
    form.supplier_id.choices = [(s['id'], s['name']) for s in Supplier.get_all()]

    if form.validate_on_submit():
        # Record stock in transaction
        item_id = form.item_id.data
        quantity = form.quantity.data
        supplier_id = form.supplier_id.data
        date_received = form.date_received.data.strftime('%Y-%m-%d %H:%M:%S') if
form.date_received.data else None

        StockTransaction.record_stock_in(item_id, quantity, supplier_id, date_received)
        flash('Stock received successfully', 'success')
        return redirect(url_for('stock_in'))

    # Get stock in history
    history = StockTransaction.get_stock_in_history()

    return render_template('stock_in.html', form=form, history=history)

@app.route('/stock/out', methods=['GET', 'POST'])
def stock_out():
    form = StockOutForm()

    # Populate select fields
    form.item_id.choices = [(item['id'], f"{item['name']} (Stock:
{Item.get_current_stock(item['id'])})")
                            for item in Item.get_all()]
```

```
if form.validate_on_submit():
    # Check if enough stock is available
    item_id = form.item_id.data
    quantity = form.quantity.data
    current_stock = Item.get_current_stock(item_id)

    if current_stock < quantity:
        flash(f'Not enough stock available. Current stock: {current_stock}', 'danger')
    else:
```

CONCLUSION:

The Inventory Management System is a comprehensive web-based application designed to streamline and centralize stock, supplier, and transaction management. Leveraging the Flask web framework, the backend efficiently handles routing, database interactions, and role-based access control, while the frontend—enhanced with Bootstrap—ensures a responsive and user-friendly interface. Core features include real-time stock updates, detailed reporting, supplier tracking, and user administration. The application promotes data integrity and operational efficiency through dynamic form handling, filtered reporting, and visual analytics. Scalable and modular, the system is well-suited for educational projects as well as small to mid-sized enterprise deployment.

REFERENCES:

1. **Flask Documentation**
   Flask: Web Development with Python
   https://flask.palletsprojects.com/

2. **SQLite Documentation**
   SQLite: Lightweight Database Engine
   https://www.sqlite.org/docs.html

3. **Bootstrap Framework**
   Front-End Framework for Responsive Web Design
   https://getbootstrap.com/

4. **WTForms Documentation**
   Flexible Forms Validation and Rendering
   https://wtforms.readthedocs.io/

5. **Jinja2 Template Engine**
   Modern and Designer-Friendly Templating for Python
   https://jinja.palletsprojects.com/